



Name: \_\_\_\_\_ Cohort/Batch: \_\_\_\_\_ Date: \_\_\_\_\_ Score: \_\_\_\_\_

KOA PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on async middleware, ctx object, routing, and error handling

TOTAL: 10 QUESTIONS

**Q1** What is Koa and how does it improve on Express's middleware model?

\_\_\_\_\_

\_\_\_\_\_

**Q6** How does centralized error handling work in Koa? **Observability**

\_\_\_\_\_

\_\_\_\_\_

**Q2** How does Koa's cascading middleware work in practice?

\_\_\_\_\_

\_\_\_\_\_

**Q7** How do you parse request bodies in Koa? **Lifecycle**

\_\_\_\_\_

\_\_\_\_\_

**Q3** What is the Koa ctx (context) object? **Configuration**

\_\_\_\_\_

\_\_\_\_\_

**Q8** How do you manage sessions in Koa? **Compliance**

\_\_\_\_\_

\_\_\_\_\_

**Q4** What is the difference between ctx.body and ctx.response.body? **Hardening**

\_\_\_\_\_

\_\_\_\_\_

**Q9** How do you serve static files in Koa? **Testing**

\_\_\_\_\_

\_\_\_\_\_

**Q5** How do you handle routing in Koa with koa-router? **SSO / IdP**

\_\_\_\_\_

\_\_\_\_\_

**Q10** How do you test a Koa application? **Tuning**

\_\_\_\_\_

\_\_\_\_\_



## ANSWER KEY &amp; EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls &amp; reference

VERIFIED SOLUTIONS

**A1 Koa is built by the Express team**

Mitigation

Koa is built by the Express team using `async/await` instead of callbacks. Its cascade middleware uses `async` functions and `await next()`, giving you clean before/after control without nesting. Koa has no bundled routing or body parsing use separate packages.

**A6 Add an error-catching middleware as the first**

Observability

Add an error-catching middleware as the first in the stack: `try { await next() } catch (err) { ctx.status = err.status || 500; ctx.body = {error: err.message}; ctx.app.emit('error', err, ctx) }`. It catches errors from all downstream middleware.

**A2 Middleware runs sequentially until `next()` is awaited,**

Primitives

Middleware runs sequentially until `next()` is awaited, then in reverse after. Code before `await next()` runs on the way in; code after runs on the way out. This makes it trivial to measure duration: record time before `next()`, log delta after.

**A7 koa-bodyparser adds `ctx.request.body` for JSON and**

Automation

koa-bodyparser adds `ctx.request.body` for JSON and URL-encoded bodies. Register it before routes: `app.use(bodyParser())`. For multipart file uploads use `@koa/multer` or busboy-based middleware. koa-body combines both.

**A3 `ctx` bundles the Node.js request and response:**

Hardening

`ctx` bundles the Node.js request and response: `ctx.request` (Koa's enhanced request), `ctx.response`, `ctx.params` (router), `ctx.state` (app-level data store), and shortcuts like `ctx.body`, `ctx.status`, `ctx.get()`, `ctx.set()`. Middleware reads and writes via `ctx`.

**A8 koa-session adds `ctx.session` to store per-user data**

Governance

koa-session adds `ctx.session` to store per-user data in a signed cookie. For large payloads configure an external store (Redis). `app.keys = ['secret']` must be set for cookie signing. Session data is loaded on request and saved on response.

**A4 They are the same underlying property**

Gotchas

They are the same underlying property. `ctx.body` is an alias for `ctx.response.body`. Koa delegates many properties from `ctx` directly to `ctx.request` and `ctx.response` as shortcuts. Setting `ctx.body` to an object automatically sends JSON with the correct Content-Type.

**A9 koa-static(`root`, `options`) creates middleware that checks if**

Performance

koa-static(`root`, `options`) creates middleware that checks if the URL maps to a file in `root`, reads it, and sets the correct Content-Type. Register it early so static file requests bypass the router entirely. Set `maxage` for browser caching.

**A5 `const router = new Router(); router.get('/users/:id', async`**

Iteration

`const router = new Router(); router.get('/users/:id', async ctx => { ctx.body = await getUser(ctx.params.id) });` Use `router.routes()` and `router.allowedMethods()` as middleware on the Koa app. Groups can share prefixes with `router.prefix('/api')`.

**A10 Use supertest: `const server = app.callback(); await`**

Optimization

Use supertest: `const server = app.callback(); await request(server).get('/users').expect(200)`. `app.callback()` returns a plain `http.RequestListener` without starting a server, allowing tests to use ephemeral ports managed by supertest.